ROBOTS MEET ARTS

EDUCATIONAL ROBOTICS AND CODING MEET ARTS AND HUMANITIES IN INCLUSIVE LEARNING ENVIRONMENTS

MODULE 3

ER AND CODING ACTIVITIES IN ARTS AND HUMANITIES

COORDINATOR & PARTNERS

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

1. Introduction

Module’s Objectives

In this module, teachers step into the role of students and experience a programming lesson from the children's perspective. At the same time, they also analyze the teacher's role within the lesson. They explore how the lesson unfolds and analyze its structure and content. Through this process, they gain insights into the students' learning experience and the teachers role. After thise they review the project’s pre-designed lessons, which integrate technology and creativity to engage students and increased their computational thinking skills.

article: https://www.edutopia.org/article/coding-robotics-ela-classrooms/
article: https://www.smartbrief.com/original/tips-using-coding-robots-classroom

Training intention

After attending this session, you will know/be able to:

- get started with unplugged computational thinking in the classroom.
- how to Integrate Robots into Your Lesson Plan
- understand the lesson plans and adapt them to meet the specific needs of your class group.

Competencies to be developed

Future-Ready Teaching Practices

Teachers practice adapting to innovative teaching methods, increasing their confidence in using new educational technologies with in there classeroom setting.

Creativity in Education

The training encourages the integration of non programming subjects with technology, inspiring teachers to create more engaging and interdisciplinary lessons.

Digital Literacy

Teachers become familiar with programming concepts and tools and gain confidence in using them, enabling them to better guide students in digital learning environments.

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

2. Theory and key concepts

Understanding the basics

The theory of module 1 & 2 will be put to practice in module 3, by giving 2 trial lessons (1 un-plugged/ 1 plugged).

Exemples of **unplugged** lessons/activities as an introduction of CT:

- Dance your block (level: easy)
- Human robot prepares breakfast (level: easy)
- Code Collage Craze (level: easy)
- The Curry Quest (level: hard)

Exemples of **plugged,** block based, lessons/activities as an introduction of CT:

- MakeCode Arcade (LAB?)
- Getting started with Octo Studio (level: easy)
- Making faces with Octo Studio (level: medium)

After the lessons, there will be reflection on the content and competencies of CT.

- What competencies CT are addressed?
- In which lessons do you see opportunities to use programming?
- What barriers do you experience?

On Moodle, you will find a series of lessons that fit within different learning areas. We clarify the layout and parts of the template.

Co-funded by the European Union

Co funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

Subject:
text

Content:
Text

Computational practices: choose what fits

- Decomposing
- Pattern recognition
- Abstraction
- Algorithms
- Problem solving

Unplugged/plugged:

Preparation time:
xx min

Age:
x-xyears

Duration:
x min

Location:
x

Group distribution:
x

Difficulty

1 robot = level easy
2 robots = level medium
3 robots = level hard

After this lesson, the children will be able to...

text

Preparation and teacher notes

Text

Here you will find practical suggestions as well as opportunities for differentiation.

Warm up x'

Text

Build up x'

Text

Ready to use lesson plans

Introducing lessons:

You can use these lessons as a starting point to familiarize your students with the core concepts of Computational Thinking (CT), gradually building their skills and confidence.

- 4 unplugged lessons
- 3 block-based lessons

Coding lessons for arts and humanities subjects:

Each subject area includes three ready-to-use lesson plans: one unplugged activity (no devices required), one using a block-based coding tool, and one lesson featuring a floor robot to support hands-on learning.

- 3 Geography
- 3 Foreign languages
- 3 Arts & Design - Art history
- 3 Civic & Moral Education
- 3 Mother Language
- 3 Music Education
- 3 History

By integrating robotics into different subject areas, we can motivate children to engage with CT. These skills are also highly relevant in daily life, where technology and problem-solving increasingly go hand in hand.

Explanation of the tools used in the lesson plans will be given in module 5.

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

3. Inclusiveness applied to this module

Introduction to the inclusiveness challenges

The lesson plans are standard, but each school and class has its own needs. It is up to the teacher to find the needs of the class group and adjust the lesson plans to meet them. This can be challenging. Therefore, they will be supported in choosing an appropriate lesson plan. Then they will proceed with that lesson plan by making small adjustments that will have a big effect on their own class group. They then test this lesson plan in their own class.

4. Module Assignment

Planning with All Learners in Mind

Select one of the provided lesson plans that you find suitable and would like to implement in your own classroom practice. Once you've made your choice, consult the 'List of Easy-to-Implement Practices to Reduce Inclusiveness issues' and consider how these strategies can be integrated into your lesson. By consciously applying these inclusive practices, you help create a more accessible and engaging learning experience for all students. This not only supports smoother classroom management but also ensures that every child can participate meaningfully and enjoy the lesson.

Questions that may help in selecting a lesson plan:

Content Suitability:
- Does the lesson content align with the age, level, and interests of your students?

Prior Knowledge:
- Are your students already familiar with concepts such as computational thinking, educational robotics, or the programming language used in the lesson?

Differentiation:
- Do any adjustments need to be made to accommodate diverse learning needs (e.g. gender differences, learning disabilities, multilingual backgrounds)?

Materials Availability:
- Are all required materials available in your classroom, or do any need to be purchased or borrowed in advance?

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

List of easy-to-implement practices for lowering inclusiveness issues

Universal Design for Learning (UDL) and computational thinking (CT) can reinforce each other in education. UDL ensures that lessons are accessible to all students, while CT teaches them to solve problems like a computer. By applying UDL in computational thinking lessons, students can learn, express themselves and become involved in the learning process in different ways, regardless of their background or learning style.
The application of UDL in CT lessons is not so different from other areas of learning.

Below you can find a list of possible strategies to support you in making your lesson more inclusive. Select what is applicable to your class and implement it in your chosen lesson.

Differentiation in content:

- Draw real-world connections. This will increase student engagement and make lessons more meaningful.
- Provide step-by-step guids, templates, and exemplars to support students who need more structure in the robotics and coding tasks.
- Reflect and iterate.
- Create tasks with varying levels of difficulty that address the same fundamental concept but allow for differtiation based on skill level. The lesson plans always indicate the level using 1, 2 or 3 robots, ranging from easy to medium to difficult. Always start with the easiest level. You can also offer different lessons with the same goal to different groups in the classroom.

Differentiation in learning profile:

- Use a mix of direct instruction, guided practive, and independent study to cater to different learning preferences.
- Rotate students through different groups based on the task or project phase, allowing them to work with peers of similar or varied skill levels.
  Attitudes that support computational thinking:
  - Confidence to tackle complex issues.
  - Perseverance in the face of difficult problems.
  - Dealing with ambiguity and open-ended problems.
  - Collaborating and communicating to achieve common goals.
  Sub-competencies in computational thinking:
  - Formulating problems.
  - Organising and analysing data logically.
  - Presenting data through abstraction.
  - Automating solutions with algorithmic thinking.
  - Identifying, analysing and implementing efficient solutions.
  - Generalising and applying the solution process to other problems.

Co-funded by the European Union
Co funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

- CT education is suitable for all pupils of all levels. The more a child is stimulated, the higher their level of development will be.

Differentiation in language level:

- Foreign-language or multilingual pupils have an extra need for visual support and step-by-step instructions. When texts or assignments need to be read, it is best to group these pupils together with pupils who speak the native language.
- You can offer simpler texts to pupils who are less proficient at reading comprehension.
- Pay sufficient attention to new concepts such as abstraction, pattern recognition, etc. In addition to sufficient repetition, it may also be useful to provide a glossary or encyclopaedia containing the vocabulary.
- Give all pupils sufficient opportunities to speak, including those who are learning the language and those who are unsure of themselves. Start with small groups to lower the threshold and increase confidence in speaking.

Differentiation in choice:

- Embrace student choice.
- Allow students to choose from a range of project topics or final products, such as a written report, a video demonstration, or a live presentation.
- When you can inspire wonder in your students, they are more motivated to work with robots. Encourage children to ask questions and engage in investigative work.
- Girls often have less confidence when it comes to CT. However, girls are just as curious as boys when it comes to computers and robots. It is important to involve girls as much as boys in the classroom, without prejudice. Ask both equally difficult questions. When you link CT to different subject areas, you can also connect more closely with the girls' everyday lives. Choose the appropriate topic from our range.

Differentiation in pace:

- Embrace mistakes.
- Collaborative projects: Scaffolded learning
- Use regular check-ins quizzes and informal observations to monitor student progress and adjust instruction as needed.

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

Computational thinking goals by age group

Goal	6-8y	9-10y	11-12y
Working in a step-by-step and goal-oriented manner.	- The pupils can carry out the steps of the CT project in pairs under the guidance of the teacher, taking into account the predefined criteria. - The pupils can work with the aid of a visual demonstration, working step by step, or with the aid of a visual step-by-step plan.	- The pupils can carry out the steps of the CT project in pairs under the guidance of the teacher, taking into account the predefined criteria. - The pupils can work with the aid of a visual demonstration, in which several steps can be presented together, or with the aid of a visual step-by-step plan.	- Students can carry out the CT project individually and in pairs under the guidance of the teacher, taking into account the set criteria. - Students can work with the help of a visual demonstration, in which multiple steps can be presented together, or with the help of a visual step-by-step plan.
Formulate the problem/question	- Talk about how they solved a "problem" (using a computer). - Name the purposes for which they use a smartphone, tablet or computer. - Name instances where a computer influenced their decision-making. - Talk about instances where the use of a computer added value.	- Discussing how "problems" are solved with a computer outside the classroom environment. - Exploring the possibilities within a given context to solve your own problems with a computer. - Expressing a (sub)problem in your own words to understand it better. - Talking about decisions made and solutions found (e.g. What has been the added value of the computer?).	- Take a position on how "problems" are solved with a computer. - Discuss advantages and disadvantages of using computers in a given situation or context. - Explore options for solving problems with a computer. - Engage the computer as a means to solve a (given) problem. Unplugged: articulate a given task at their own level.

Co-funded by the European Union

Goal	6-8y	9-10y	11-12y
Developing CT	Decomposition - Pupils can divide a task into subtasks. - Place (sub)tasks in a logical order. - Divide a concrete problem into subtasks.	Decomposition - Pupils can divide a more complex task/process into subtasks. - Pupils can experience specific forms of data representation such as Morse code or secret writing. - Pupils are familiar with the binary number system. - Pupils can represent data in a simple graph.	Decomposition - Pupils can divide a more complex task/process into subtasks. - Pupils are familiar with the binary number system. - Pupils can display data in a simple graph. - Check whether any important steps are missing or forgotten when performing a sub-task.
	Pattern recognition - Pupils can give examples of a pattern/automated system in everyday life. - Pupils can make “if…then” arguments. - Recognise the repetition of tasks in different situations. This includes tasks for which a computer is used.	Pattern recognition - Pupils can give examples of patterns in everyday life and in abstract situations. - Pupils can make “if…then” arguments. - Check whether patterns can be simplified. - Investigate how (a part of) a computer programme can support a recurring task or action.	Pattern recognition - Pupils can give examples of patterns in everyday life and in abstract situations. - Pupils can make “if…then” arguments. - Check whether patterns can be simplified. - Investigate how (a part of) a computer programme can support a recurring task or action.

L.A.B STIMMULI for social change UNIVERSITAT ROVIRA i VIRGILI UCLL HOGESCHOOL tWA. de ark basisschool
Co-funded by the European Union
Co-funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

Goal	6-8y	9-10y	11-12y
Developing CT	Abstraction - Pupils can recount concrete situations in broad terms. - Pupils recognise important elements in a process, story or photo. - Pupils can create an algorithm using icon blocks. - Pupils can name the pseudocode for an icon block.	Abstraction - Pupils can recount concrete situations in broad terms. - Pupils recognise and use various abstract representations such as a map or concept. - Pupils can create an algorithm using pictogram blocks and/or word blocks. - Pupils can name the pseudocode for a pictogram/word block. - Pupils can create a variable under supervision.	Abstraction - Pupils can recount concrete situations in broad terms. - Pupils can explain how devices and digital instruments work by presenting the main points. - Pupils can create an algorithm using word blocks. - Pupils can name the pseudocode for a word block. - Pupils can create a variable under supervision.

Co-funded by the European Union

Co funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.

Goal	6-8y	9-10y	11-12y
Developing computational thinking skills	Algorithms and procedures - Pupils can name the difference between hardware and software. - Pupils can name the difference between input and output components. - Pupils can use the motor, colour sensor and LED matrix in an algorithm. - Pupils can place instructions in a logical order. - Pupils can predict the outcome of a series of instructions. - Pupils can explain what an algorithm is. - Pupils can describe the characteristics of a good instruction. - Students can create multiple programming stacks side by side. - Students can create/test/debug an algorithm to solve a problem, using: sequences, events, simple repetitions (loops), simple conditions ("if...then").	Algorithms and procedures - Pupils can investigate the underlying rule for an instruction. - Pupils can name the difference between hardware and software. - Pupils can name the difference between input and output components. - The pupils can use the motor, colour sensor, distance sensor, tilt sensor and force sensor in an algorithm. - The pupils can place instructions in a logical order. - The pupils can predict the outcome of a series of instructions. - The pupils can explain what an algorithm is. - The pupils can describe the characteristics of a good instruction. - Students can create multiple programming stacks side by side. - Students can create/test/debug an algorithm to solve a problem, using: sequences, events, repetitions (loops), conditions ("if...then"), variables (to store/modify data).	Algorithms and procedures - Pupils can investigate the underlying rule for an instruction. - Pupils can name the difference between hardware and software. - Pupils can name the difference between input and output components. - The pupils can use the motor, colour sensor, distance sensor, tilt sensor and force sensor in an algorithm. - The pupils can place instructions in a logical order. - The pupils can predict the outcome of a series of instructions. - The pupils can explain what an algorithm is. - The pupils can describe the characteristics of a good instruction. - Students can create multiple programming stacks side by side. - Students can create/test/debug an algorithm to solve a problem, using: sequences, events, repetitions (loops), conditions ("if...then"), variables (to store/modify data).

5. Assessment and Reflection

Criteria for assessing the concepts and competencies acquired in this module

After attending this session, you will know/be able to:
- get started with unplugged computational thinking in the classroom?
- how to Integrate Robots into Your Lesson Plan?
- understand the lesson plans and adapt them to meet the specific needs of your class group?

Self-assessment tool applied to the module

link to the survey

6. Bibliography

Rohaan, R. e.a.. (2022). Onderzoekend en ontwerpend de wereld ontdekken. uitgeverij Noordhoff, Groningen / Utrecht

Smekens, E. (2017). De klas van (nu). Uitgeverij Acco, Leuven

van den Berg, L. and Ros, A. (2021). Begeleiden van actief leren. Uitgeverij Coutinho, Bussum

https://drive.google.com/file/d/1sp04F8UK-1yOnaP5fChCVsEEGDI2vhMV/view

https://www.edutopia.org/article/coding-robotics-ela-classrooms/

https://www.smartbrief.com/original/tips-using-coding-robots-classroom

Co-funded by the European Union

Co-funded by the European Union (Project number: 2023-1-FR01-KA220-SCH-000151881). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or of the European Education and Culture Executive Agency (EACEA). Neither the European Union nor the granting authority can be held responsible for them.